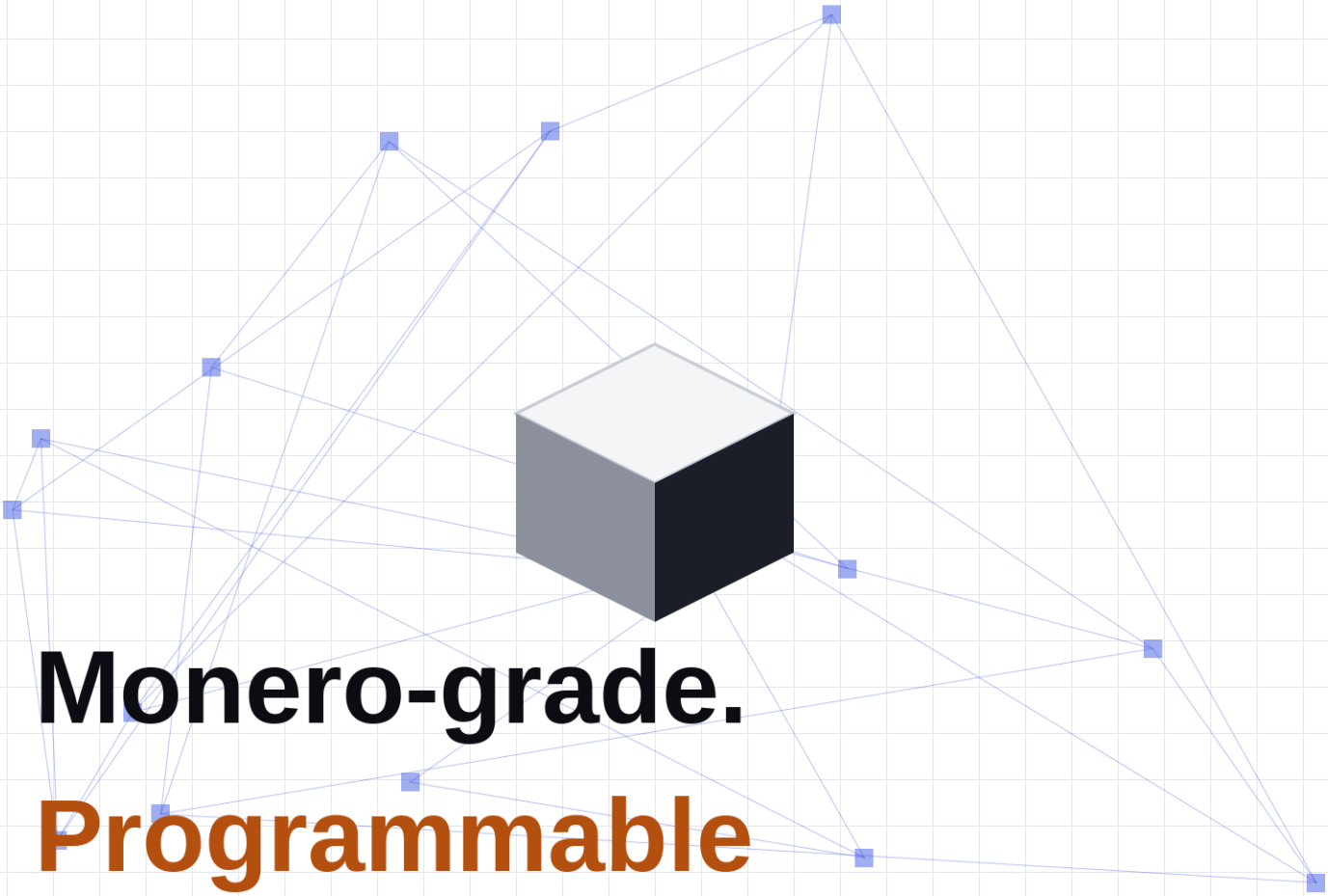
SERAPH.

ALPHA · BASE SEPOLIA



Monero-grade. Programmable private execution.

Seraph Protocol · Technical Manual

A Privacy Continuum protocol: Monero-inspired base layer, two private EVMs, shared anonymity sets, and first-party privacy modules.

RESEARCH & TESTNET

UNAUDITED PROTOTYPE

CORE CIRCUITS & PRIVACY POOLS IN DEVELOPER TESTING · AUDITS PENDING

seraphprotocol.tech · Docs edition · Mon Oct 26, 2026

Contents

About this edition 3

Status and scope 3

Risk factors and disclaimers 4

Deployment registry 5

Chapter 0: Protocol overview and the privacy continuum 6

Chapter 1: Monero heritage and evolution 8

Chapter 2: Seraphis, Jamtis, and research horizons 9

Chapter 3: Formal cryptography and primitives 10

Chapter 4: ZK circuit architecture and prover pipeline 12

Chapter 5: Base layer protocol and state machine 14

Chapter 6: Dual private EVMs 15

Chapter 7: Bridges and cross-chain architecture 17

Chapter 8: Relayers and gas abstraction 18

Chapter 9: Sequencer architecture and MEV shielding 20

Chapter 10: Wallets, scanning, and key recovery 21

Chapter 11: First-party privacy modules 22

Chapter 12: Ecosystem, DEX, and CEX integrations 24

Chapter 13: SER tokenomics, supply, and burns 25

Chapter 14: Presale architecture and vesting 27

Chapter 15: Launchpad mechanics and creator playbook 28

Chapter 16: Decentralized governance and private ballots 30

Chapter 17: Security model, threat vectors, and test coverage 31

Chapter 18: Emergency procedures and circuit breakers 33

Chapter 19: Protocol roadmap and milestones 35

Chapter 20: Glossary 36

Chapter 21: Contributing, standards, and dev setup 38

Appendix: Bibliography 39

About this edition

This is the docs edition of the Seraph Protocol Technical Manual, dated October 26, 2026. It replaces the single-PDF manual and is organized as a book with chapters and pages, so it can be imported into BookStack and kept in version control.

What changed from the original manual

- **Status on every chapter.** Each chapter states what is deployed today on the Base Sepolia testnet and what is target architecture.
- **Corrections from internal review.** Sell-tax logic, the commitment scheme wording, pause authority, the verifier interface example, the nullifier and AssetId claim, and phase numbering have been fixed or clearly flagged.
- **Decisions marked as under review.** Where the team has not yet decided something, the page says so instead of stating a settled answer. These are marked with the **Under review** label.
- **New sections.** Status and scope, risk factors and disclaimers, a deployment registry, audit status, pause coverage, and a bibliography.
- **Diagrams in the site's visual style.** Figures were redrawn to match the public site.

How to read it

Chapter numbers match the original manual, so cross-references such as "Chapter 12" still point to the same material.

Three callout labels are used throughout:

> **Status.** What is live or deployed today.

> **Under review.** A decision or fact the team has not finalized. Do not rely on the stated value until this label is removed.

> **Warning.** A risk that could cause loss of funds, privacy, or access.

Conventions

- Amounts are in SER unless marked otherwise. SER has 18 decimals.
- Addresses are shown in checksummed form. Confirm any address against the deployment registry before you send funds.
- Code samples are illustrative unless a page says the code is production code. Check the repository for the authoritative source.

Status and scope

Seraph is in active research and development. The current software is a testnet prototype. It has not been audited, and it should not be used to hold value you cannot afford to lose.

Current versus target

AREA	CURRENT (BASE SEPOLIA TESTNET, OCTOBER 2026)	TARGET
Core contracts	Test deployments on Base Sepolia (chain ID 84532). Addresses are in the deployment registry.	Mainnet deployment after audits and legal review.
Privacy pool	Single pool design in testing. Depth-20 tree, 2^20 capacity.	Audited pool, then Level 1 and Level 2 shielding for SER.
Base layer	Single-node proof of concept with in-process peer-to-peer networking.	Multi-node sovereign base layer with a hybrid consensus engine.
Seraph EVM	Not yet a separate chain. Contracts run on Base Sepolia with ETH gas.	Separate chain with SER gas.
XMR EVM	Design stage.	Testnet, then mainnet, with XMR fees through relayers.

Cross-EVM bridge	Design and testnet contracts.	Bonded relayers with a challenge window, subject to the open items in Chapter 07.
Shared anonymity set	Each pool provides its own anonymity set.	Notes from the base layer and both EVMs contributing to common pools where cryptographically feasible.
Audits	None completed.	Proposed: two independent audits and formal verification of core circuits before mainnet funds are accepted.
SER token	Test deployment only.	Mainnet token generation event (TGE) after the gates in Chapter 19.

Under review. The “shared anonymity set” row describes a goal. Full shared-set guarantees across heterogeneous environments are a research objective and are not shipped. Do not treat them as a current guarantee.

What “live” means in this manual

When a page says a feature is live, it means the feature runs on the public Base Sepolia testnet. It does not mean the feature has been audited, that it is secure against all attacks, or that it will be carried to mainnet unchanged.

Scope

This manual covers the protocol design, its cryptography, the contracts and base layer as designed, the token economics as proposed, and the developer tooling. It does not cover investment decisions, legal advice, or tax advice.

Risk factors and disclaimers

Under review. This section was drafted for review by counsel. It describes risk categories. It is not legal advice and it is not a complete list of risks.

Nothing in this manual is an offer to sell securities, an investment recommendation, or a promise of any outcome. Read the full documentation and consult your own legal, tax, and financial advisers before you interact with the protocol.

Testnet status

The contracts in this manual run on Base Sepolia, a test network. No mainnet deployment has occurred. Addresses, parameters, features, and timelines may change, and test tokens have no value.

Smart contract risk

The contracts may contain bugs. Passing internal tests does not prove the absence of vulnerabilities. See the audit status table in Chapter 17. Funds may be lost through exploits, misconfiguration, or failures in external dependencies.

Cryptographic and proving risk

The protocol uses zero-knowledge circuits, proving systems, and a trusted setup for Groth16. Bugs in circuits or verifiers could allow counterfeit assets or double spends. Cryptographic assumptions may be weakened by future research, including advances in quantum computing.

Privacy limits

Privacy depends on usage patterns and on metadata the protocol does not control. This includes network traffic, timing, deposits from identifiable sources, gas funding, wallet software, and transaction amounts. The protocol cannot guarantee anonymity.

Governance and administrator risk

Some addresses hold powers, including pausing contracts, setting tax exemptions, and setting gas rates. Depending on the final design, they may also replace verifier contracts. A compromise, error, or misconduct by these parties could harm users and token holders. See Chapter 18 for the authority map.

Bridge and custody risk

The cross-chain bridge depends on relayers and a challenge window. Bridged wXMR depends on custody arrangements and reserve verification that are still under review. Assets held in bridge or vault contracts may be lost or frozen.

Token economics

The sell tax, burn, vesting unlocks, and concentrated allocations may affect the price and liquidity of SER. Vesting schedules are measured from the TGE, which has not yet occurred. Token prices may fall below presale or market prices.

Liquidity and listings

Exchange listings, DEX liquidity, and launchpad graduation are not guaranteed. Trading may be thin, and slippage can be significant.

Regulatory and legal

Laws on privacy-enhancing technology, sanctions, know-your-customer rules, tax reporting, and token classification vary by jurisdiction and change over time. Some features or sales may be restricted or prohibited where you live. Access may be limited for sanctioned persons and jurisdictions.

Operational risk

The protocol cannot recover seed phrases or private keys. Blockchain transactions are generally irreversible. Development timelines, roadmap phases, and team capacity may change.

Tax

You are responsible for taxes that apply to your transactions. Chapter 10 describes disclosure tools that help with reporting. They do not guarantee compliance with any tax authority’s requirements.

Deployment registry

Confirm any address against this table before you send funds or integrate. Addresses marked **Pending** have not been published yet.

Base Sepolia (testnet, chain ID 84532)

CONTRACT	ADDRESS	STATUS
Presale (`Presale.sol`)	`0x2c4156e4bFE0bE6A4e683C7Be521E1cC10B798bE`	Testnet
Launchpad (`Launchpad.sol`)	`0xe0349E4cC431329207e98C5205062776e058A846`	Testnet
SeraphToken (`SERAPH.sol`)	Pending	Testnet
PrivacyPool	Pending	Testnet
CrossEVMBridge	Pending	Testnet
GasRelayer	Pending	Testnet
InvoiceRegistry	Pending	Testnet
UsageVault	Pending	Testnet

CollateralVault	Pending	Testnet
Spend verifier	Pending	Testnet

Other networks

ITEM	ADDRESS	NETWORK	STATUS
Treasury	`0xa287FC2B8f89023eF1c82af09aee351cCdDa8a76`	**Under review.** The original manual did not state the network.	Confirm before publishing.
Guardian Safe (4-of-7)	Pending	Pending	Pending
USDC (presale payment)	Pending	Target network	Pending
USDT (presale payment)	Pending	Target network	Pending

Warning. Do not send funds to an address that is not listed here, even if it appears in a third-party listing or chat message.

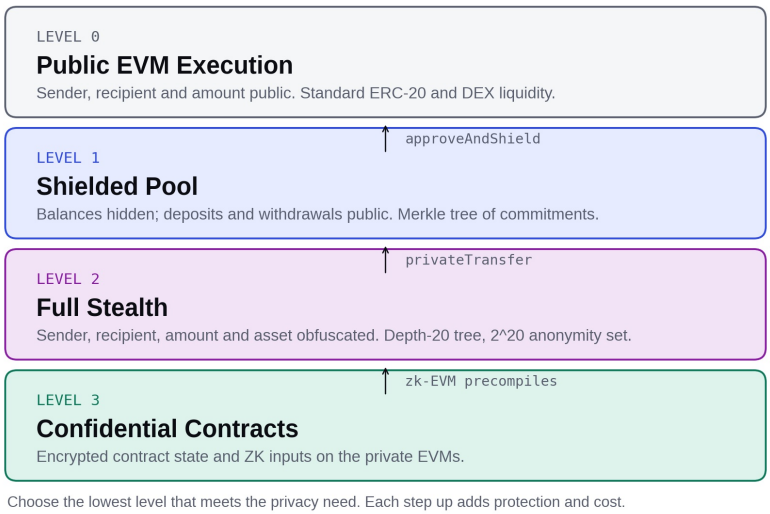
Chapter 0: Protocol overview and the privacy continuum

Seraph is a modular privacy protocol built to resolve a long-standing trade-off between three properties: cryptographic privacy, programmable smart contracts, and regulatory auditability.

Until now, decentralized ledgers have split into two camps.

1. **UTXO privacy networks, such as Monero.** They offer large anonymity sets, confidential amounts, and unlinkable payments. They lack general-purpose contract execution and multi-token composability.
2. **Account-based smart contract platforms, such as Ethereum and its rollups.** They offer composability, DeFi, and application development. They expose full transaction graphs, balances, and interaction histories to public analysis and to maximal extractable value (MEV) bots.

Seraph combines the two. A Monero-inspired base layer provides settlement and a shared anonymity pool. Two confidential EVMs, the Seraph EVM and the XMR EVM, provide programmability. Shared components keep the layers in one privacy set where cryptographically feasible.



The privacy continuum: four levels from public execution to confidential contract state

Privacy is a continuum

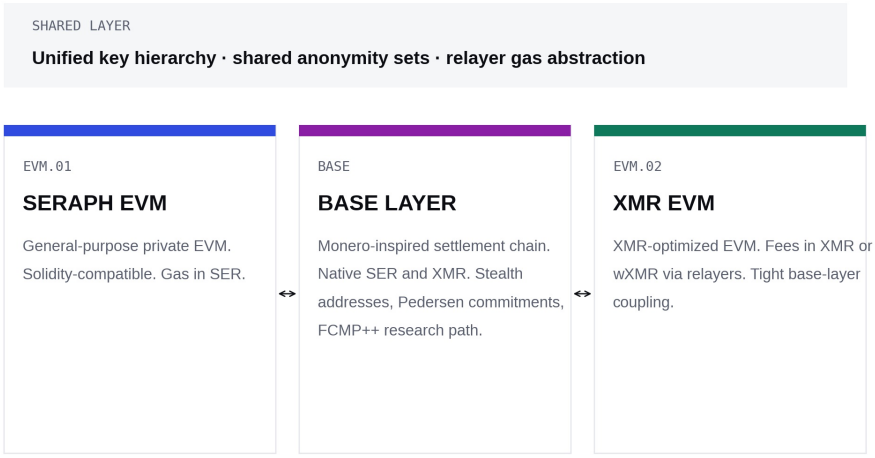
Different economic activity needs different privacy. Seraph therefore defines four levels rather than a single on/off switch.

LEVEL	NAME	WHAT IS VISIBLE	STATE ARCHITECTURE	TYPICAL USE
0	Public EVM	Sender, recipient, and amount are public	Standard ERC-20 and contract storage	DEX liquidity, launchpad discovery, public treasury accounting
1	Shielded pool	Balances are hidden. Deposits and withdrawals are public.	Merkle tree of commitments (`PrivacyPool.sol`)	Long-term balance protection
2	Full stealth	Sender, recipient, amount, and asset are obfuscated	Depth-20 incremental Merkle tree (2^20 capacity) plus nullifier registry	Private payments, payroll, dark-pool orders
3	Confidential contracts	Contract logic runs on encrypted state with zero-knowledge inputs	Dual private EVMs with homomorphic precompiles and proof verifiers	Private lending, usage billing, sealed-bid ballots

Choosing a level

- **Level 0** when you need public liquidity on automated market makers, when you interact with third-party contracts such as Uniswap v3 or Aave, or when you need to prove solvency publicly.
- **Level 1** when you want to store capital without exposing your balance to targeted phishing or balance scraping, while staying in the EVM ecosystem.
- **Level 2** when two counterparties settle sensitively and neither should be linked to the other's transaction graph.
- **Level 3** when logic is stateful and involves several parties, such as streaming payments to autonomous agents or institutional credit facilities.

Architecture at a glance



Three layers and a shared key hierarchy

The base layer settles value. The two EVMs run programmable logic and differ in gas token and fee model. The bridge moves value between them. The shared layer covers key derivation, note pools, and gas abstraction through relayers.

> **Status.** Test contracts run on Base Sepolia today. The base layer and both EVMs are target architecture. See Chapter 19 for the staged plan.

Who this manual is for

1. **End users and privacy advocates.** Shielding and unshielding, wallet safety, and selective disclosure through view keys.
2. **Smart contract and frontend developers.** Solidity contracts, the TypeScript SDK, and precompiles for confidential state, stealth addresses, and proof verification.
3. **Validators, relayers, and node operators.** Running nodes, bonded relayers on the cross-EVM bridge, and sequencer clusters.
4. **Token holders and ecosystem creators.** SER tokenomics and supply, governance, presale mechanics, and launchpad token creation.

Chapter 1: Monero heritage and evolution

Monero showed that decentralized money can be fungible by hiding senders, recipients, and amounts. Seraph inherits that design philosophy and changes the underlying machinery to support higher throughput and general-purpose contracts.

Ring signatures and their limits

Monero's signatures evolved from LSAG and MLSAG to CLSAG. In a ring signature, the real signer mixes an input with decoy outputs. An observer knows that one ring member spent the output but cannot tell which one. In current Monero the ring size is fixed at 16.

Ring signatures have three limits:

1. **Bounded anonymity set.** Sixteen possible signers per input.
2. **Statistical clustering.** Chain analysis uses timing and spend-history heuristics, which reduce the effective ring size when known outputs are spent.
3. **Linear footprint.** Signature size grows with ring size, O(n). Larger rings would make verification too costly for consumer hardware.

The Seraph shift: incremental Merkle trees and zero-knowledge proofs

Seraph replaces decoy rings with full-membership proofs over an incremental Merkle tree.

- **Anonymity set.** Every unspent note in the tree. The tree holds 2^20 = 1,048,576 notes per pool.
- **Proof size.** Constant-size Groth16 or PLONK proofs of about 128 to 256 bytes, verified in roughly 3 ms on-chain.
- **No decoys.** The proof shows the spent note is somewhere in the accumulated tree without revealing which leaf.

Stealth addresses

Seraph uses the dual-key stealth address protocol (DKSAP). A recipient publishes a scan public key A = aG and a spend public key B = bG. A sender picks an ephemeral scalar r and publishes R = rG. Both sides compute the shared secret, which is rA = aR. The sender derives the one-time destination P = H_s(rA)·G + B. Only the recipient, who knows a and b, can recognize the payment and recover the spend key x = H_s(aR) + b. Chapter 3 gives the full derivation.

Confidential amounts

Pre-RingCT, Monero exposed amounts. RingCT introduced Pedersen commitments, C = vH + rG, which hide the value while keeping the balance check possible. Range proofs prevent negative amounts from creating value.

Bulletproofs

Monero moved from multi-kilobyte Borromean range proofs to Bulletproofs, which reduced transaction size by more than 80% using logarithmic inner-product arguments. Bulletproofs++ further reduces proof size and verification time. Seraph builds range checks into its arithmetic circuits so that verification happens inside the EVM with no separate proof system.

Key images and nullifiers

Monero derives a key image I = x·H_p(P) from the one-time private key. The same note always produces the same image, so a second spend is detected.

Seraph's EVM pool uses a deterministic nullifier, N = Keccak256(spendKey ‖ leafIndex). Revealing N marks the note spent without revealing which note it was. Lookup in the nullified mapping is constant time.

Full-chain membership (FCMP++)

Monero research is pursuing full-chain membership proofs (FCMP++), which replace decoy rings with polynomial commitments such as Curve Trees over the whole chain history.

> **Status.** Seraph's EVM pool already provides full-membership proofs over a depth-20 tree. FCMP++ and Curve Trees are a research path toward trustless recursion between Curve25519 and BN254. They are not in the current release.

Chapter 2: Seraphis, Jamtis, and research horizons

Production architecture versus research horizons

> **Status.** The current deployment includes the Base Sepolia contracts, the Rust base layer, and the TypeScript SDK. It uses an incremental Merkle tree of depth 20 (2^20 notes), deterministic nullifiers, and Groth16 or PLONK spend verification.
>
> Seraphis, Jamtis, and FCMP++ are next-generation upgrade paths on the long-term research roadmap. They are not active in current contracts. They form the design basis for the research phase of the roadmap in Chapter 19.

Seraphis: enotes and separation of concerns

Seraphis is a transaction protocol designed for Monero by Koe. It replaces the single UTXO concept with modular objects called enotes. In legacy CryptoNote-style designs, one private key handles three jobs: detecting incoming funds, proving balance ownership, and authorizing spends. Seraphis separates them into distinct keys: a spend key that authorizes outflows, a view key that audits amounts, and a scan key that identifies inflows.

An enote contains three parts:

1. A value commitment that hides the amount.
2. A one-time public key, derived through a stealth address protocol.
3. A key-image generator, used only for nullifier creation and separated from the one-time address.

Jamtis: hierarchical addresses and wallet keys

Jamtis is the addressing standard designed to accompany Seraphis. Its address is a tuple of spend, view, and scan components. This structure allows three practical features:

- **Fast scanning.** A light client or hardware wallet can delegate the scan key to a cloud scanning service without giving that service the ability to see values or spend funds.
- **Selective audit keys.** An auditor can hold a read-only key that reveals amounts for tax reconciliation but cannot initiate transfers.
- **Delegated proof generation.** Proof generation can be offloaded to a proving service using an ephemeral delegation key, with the privacy trade-offs described in Chapter 11.

Balance and membership proofs

A Seraphis transaction is an algebraic balance proof with three parts: membership proofs for each input, a value balance proof, and range proofs for each output.

How Seraph adapts these principles

Seraph takes two ideas from Seraphis: modular key separation and unlinkable enote structure. It adapts them to an EVM-friendly design.

1. **Dual view keys in production.** The SDK derives an incoming view key for scanning and an outgoing view key for generating compliance disclosures. Chapter 10 gives the derivation.
2. **Note commitments in an indexed tree.** Inclusion proofs can be verified on-chain in constant time.
3. **Native multi-asset isolation.** Each note carries an asset domain tag. SER, XMR, and custom assets can share one anonymity pool without the risk of substitution across assets.

> **Under review.** The base-layer note commitment is defined as a Keccak256 hash over the asset ID, amount, recipient key, and blinding factor. Balance conservation is proven inside the spend circuit. Chapter 3 describes Pedersen commitments and a homomorphic balance check. The team is deciding which commitment the circuit uses. Until that is settled, treat the commitment description in Chapter 3 as the design intent and check the code.

Chapter 3: Formal cryptography and primitives

Curves

Seraph uses two elliptic-curve groups.

1. **Ed25519 and Curve25519** on the base layer, for stealth addresses, signatures, and view-key encryption. The field prime is p = 2^255 − 19.
2. **BN254 (alt_bn128)** for on-chain SNARK verification through the EVM precompiles at addresses 0x06, 0x07, and 0x08.

For BN254:

- Base field modulus p = 21888242871839275222246405745257275088696311157297823662689037894645226208583
- Scalar field order r = 21888242871839275222246405745257275088548364400416034343698204186575808495617

> **Under review.** The original manual labeled these two constants the other way around. Re-verify both values against EIP-197 before publication.

Notation used in this chapter:

- H_s(·) hashes a byte string to a scalar in the scalar field.
- H_p(·) hashes data to a point on the curve.
- x ←$ F_q denotes uniform random sampling from the field.

Pedersen commitments

A Pedersen commitment binds a committer to a value v without revealing it:

```
C = v·H + r·G
```

Here v is in [0, 2^64 − 1], r is a uniform blinding factor, and G and H are independent generators. No one knows log_G(H).

- **Hiding.** For any value v and any commitment C, some blinding factor r makes C valid. An observer learns nothing about v.
- **Binding.** Two different openings of the same commitment would reveal log_G(H), which is assumed infeasible.
- **Homomorphism.** $C_1 + C_2$ commits to $v_1 + v_2$ with blinding $r_1 + r_2$.

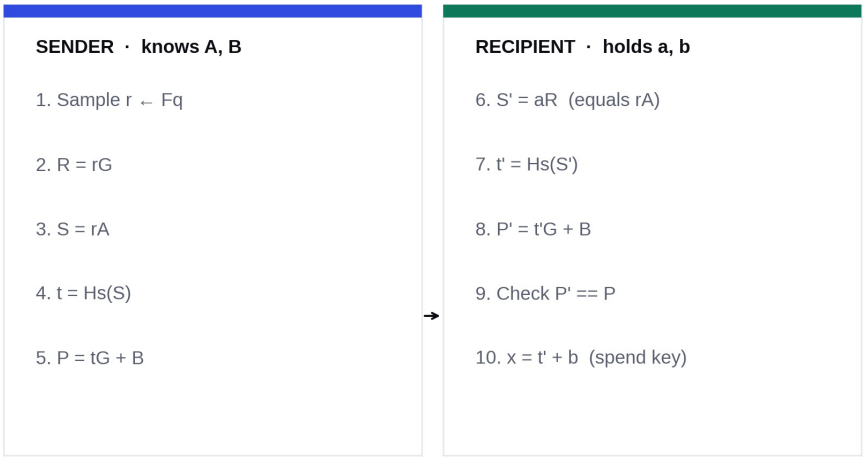
Stealth address derivation

The DKSAP flow is as follows.

1. The sender samples r, computes R = rG and the shared secret S = rA.
2. The sender computes t = H_s(S) and the one-time address P = tG + B.

3. The sender publishes (R, P, encrypted memo).
4. The recipient computes S’ = aR, which equals S. The recipient computes t’ = H_s(S’) and P’ = t’G + B.
5. If P’ equals P, the note is theirs. The one-time spend key is x = t’ + b, and xG = P.

Only the holder of the spend key b can spend from P.



The sender publishes (R, P, memo). Only the holder of b can spend from P. The scan key a alone can detect the note.

Stealth address flow between sender and recipient

Key images and nullifiers

Base layer (Monero-style): for a one-time key P = xG, the key image is I = x·H_p(P). Spending the note twice requires the same x, which gives the same I.

EVM privacy pool: each note has a nullifier N = Keccak256(spendKey ‖ leafIndex). The contract checks that N has not been used, then records it. The verifier confirms that N corresponds to the note at leafIndex in the tree.

> **Under review.** The threat matrix in Chapter 17 states that AssetId is hashed into the nullifier. The nullifier formula above does not include AssetId. Either the formula or the threat-matrix entry will change. Check the code.

Balance equation with protocol fees

For m input notes, n output notes, and public fee f:

```
Σ v_in  =  Σ v_out + f
```

In commitment form, with Δr = Σ r_in − Σ r_out:

```
Σ C_in − Σ C_out − f·H = Δr·G
```

The prover shows knowledge of Δr with respect to the public point Δr·G.

> **Under review.** This balance check assumes Pedersen commitments. Chapter 2 notes that the base-layer commitment is Keccak-based. Confirm which balance check the circuit uses before relying on this section.

Range proofs

Without range proofs, an attacker could create a negative output. In modular arithmetic, −5 is represented as q − 5, which is a huge positive number. Seraph enforces v ∈ [0, 2^64 − 1]. On the base layer this is proven with Bulletproofs++. On the EVM it is proven with range constraints inside the SNARK circuit.

Viewing keys and selective disclosure

Seraph separates the ability to see from the ability to spend.

1. **Incoming view key.** Lets a wallet scan the tree and decrypt incoming amounts and memos. It cannot spend.
2. **Outgoing view key.** Authorizes compliance packages that show the amounts, destinations, and timestamps of a user's outgoing transactions to an accountant or auditor, without giving spending authority.

Chapter 10 gives the derivations.

Chapter 4: ZK circuit architecture and prover pipeline

Proof systems

Seraph converts state transitions into rank-1 constraint systems (R1CS) and algebraic intermediate representations (AIR). Two proving backends are used.

1. **Groth16 on BN254.** Used on the EVM for gas efficiency. Proofs are about 128 bytes. Verification uses pairing checks.
2. **UltraPLONK and Halo2.** Used for more complex confidential contract state on the private EVMs. These support custom gates, lookup tables, and a universal setup.

The private spend circuit

The circuit behind `PrivacyPool.sol` enforces four constraints for each spend.

1. **Merkle inclusion.** Given a historic root R from the ring buffer of the last 100 roots and a note commitment C, the prover shows that C sits at leafIndex. Hashes are chained from C up 20 levels, and the final hash must equal R.
2. **Nullifier derivation.** The prover knows a spendKey such that N = Keccak256(spendKey ‖ leafIndex). The spend key is never revealed, so the nullifier cannot be forged.
3. **Commitment validity.** The note commitment is correctly formed from its fields.
4. **Range and conservation.** Each output value is in [0, 2^64 − 1], and Σ v_in = Σ v_out + fee.

Public signals

The verifier takes a proof and an array of public signals in this order:

INDEX	SIGNAL
0	Historical Merkle root R
1	Nullifier hash N
2	Output commitment 1
3	Output commitment 2
4	Public amount (non-zero only when unshielding)
5	Public recipient address (non-zero only when unshielding)

The verifier interface is:

```
interface IVerifier {
    function verifyProof(
        bytes calldata proof,
```

```
        uint256[] calldata pubSignals
    ) external view returns (bool);
}
```

Note. An earlier draft used the Bulletproof precompile at 0x23 as a verifier. That precompile verifies range proofs only. Spend proofs use the verifier above.

Verification gas

Design estimates from the original manual:

OPERATION	GAS (DESIGN ESTIMATE)
ecAdd (0x06)	150
ecMul (0x07)	6,000
ecPairing (0x08), 4 pairs	185,000
Total `verifyProof`	about 210,000

Under review. An earlier section gave 3 pairings and about 190,000 gas. This table uses 4 pairs and about 210,000. Replace both with a single measured figure, along with the measurement method, before publication.

Client-side proving

All zero-knowledge proofs are generated on the user's device. Private keys and blinding scalars never leave the device. The prover compiles to WebAssembly with SIMD, with experimental WebGPU shaders.

PLATFORM	ENGINE	SPEND PROOF TIME	PEAK MEMORY
Apple M2 Max, Chrome	WebGPU	1.24 s	185 MB
Apple M2 Max, Node.js	Native C++/Rust	0.88 s	140 MB
Intel Core i7-12700K	WASM + SIMD	2.15 s	210 MB
iPhone 15 Pro, Safari	WebAssembly	3.42 s	195 MB
Samsung Galaxy S23, Chrome	WebAssembly	4.10 s	220 MB

Under review. These figures need the circuit constraint count, prover version, test date, and benchmark method alongside them.

Trusted setup

Groth16 needs a structured reference string. Seraph uses the Perpetual Powers of Tau ceremony, which has many contributors. The setup is secure if at least one participant destroyed their toxic waste.

Under review. Groth16 also requires a circuit-specific phase 2 ceremony. Document who ran it, the transcripts, the verification steps, and the disposal claim. This section is incomplete until that is done.

PLONK uses KZG commitments over a universal setup.

Future STARK migration

The research roadmap plans to add transparent, quantum-resistant STARKs using hash functions such as Poseidon or Rescue-Prime. STARKs remove the trusted setup. This is a Phase 4 research target and is not part of the current design.

Chapter 5: Base layer protocol and state machine

Goals and architecture

The Seraph base layer is a custom node built in Rust (`seraph-base-layer`). It is the settlement layer and the shared anonymity pool for the ecosystem.

Unlike EVM chains, which store balances in world state, the base layer tracks state through two structures.

1. **Global commitment tree.** An incremental Merkle tree of all minted note commitments.
2. **Nullifier accumulator.** An append-only registry of consumed nullifiers.

```
SERAPH BASE LAYER NODE
├── P2P gossip (libp2p, Noise protocol, Tor/I2P routing)
├── Dual-fee engine (native SER and native XMR settlement)
├── State machine
│   ├── Incremental Merkle tree (2^20 per tree level)
│   ├── Nullifier double-spend engine
│   └── Multi-asset domain isolation (SER, XMR, custom)
└── Storage engine (RocksDB or Sled, embedded key-value)
```

Status. A single-node proof of concept with in-process networking is in progress. The multi-node design described here is the target.

Block header

Each block header contains:

- `version` : protocol version (32-bit).
- `previous_block_hash` : 32-byte hash of the parent block.
- `merkle_root` : root of the transaction inclusion tree.
- `state_root` : root of the note commitment tree after the block.
- `nullifier_root` : root of the newly consumed nullifiers.
- `timestamp` : 64-bit UNIX timestamp.
- `height` : block index.
- `nonce` : proof-of-work or validator signature, depending on network phase.

Under review. The meaning of `version` (a protocol version scheme) is not yet specified.

Consensus

- **Devnet and testnet phases.** Authority-validated sequencing with deterministic consensus for fast testing.
- **Later phases.** A hybrid proof-of-stake and RandomX-variant checkpointing engine. Native Monero miners may co-mine or validate Seraph blocks.

Under review. The consensus specification does not yet define validator selection, fork choice, finality, or slashing for the hybrid engine. Those rules are required before any mainnet launch.

Transactions and enotes

Transactions consume input notes and create output notes.

```rust
pub struct Transaction {
    pub inputs: Vec<[u8; 32]>,   // spent note nullifiers
    pub outputs: Vec<Note>,      // new confidential notes
    pub fee_ser: u64,            // fee in SER units
    pub fee_xmr: u64,            // fee in atomic XMR units
    pub public_inputs: Vec<u8>,  // Merkle roots and public inputs
```

```
        pub proof: Vec<u8>,          // zero-knowledge spend proof
    }
```

Validation rules:

1. **Nullifier uniqueness.** No input nullifier may already be in the ledger's set.
2. **Atomic consumption.** All input nullifiers are inserted in one database transaction. If any fails, the whole transaction reverts.
3. **Commitment insertion.** Output commitments are appended to the tree in order.
4. **Conservation.** Inputs equal outputs plus fees, for each asset class.

Dual-fee engine

Transactions can pay fees in SER or in XMR.

- **SER** supports the deflationary burn and treasury allocation.
- **XMR** lets Monero holders settle transactions without holding a second gas token.

When a transaction pays in XMR, its inputs consume XMR notes. Relayers or block proposers can receive fees in XMR into private stealth addresses. Assets never mix, so rounding errors cannot cross between SER and XMR.

Multi-asset domain isolation

The asset identifier is bound into the note commitment, so notes of different assets cannot be substituted for each other. Even if an attacker builds an XMR note with the same amount, recipient key, and blinding as a SER note, the commitments and nullifiers differ.

> **Under review.** Chapter 17 says the asset identifier is also hashed into the nullifier. Chapter 3 gives a nullifier formula without it. See Chapter 3.

Networking milestones

1. **Proof of concept.** In-process, tested with multi-actor harnesses that cover state transitions, mempool double-spend rejection, and ring-buffer eviction.
2. **Single-node devnet.** A local RPC node exposing JSON-RPC over HTTP and WebSocket, integrated with `@seraph-protocol/sdk`.
3. **Multi-node testnet.** A decentralized libp2p gossip mesh with Dandelion++ dissemination and built-in Tor and I2P proxies.

> **Under review.** These milestones use different phase numbers from Chapter 19. See the mapping table in Chapter 19.

Chapter 6: Dual private EVMs

Seraph plans two specialized Ethereum Virtual Machines that provide programmable privacy without giving up developer tooling.

> **Status.** The Seraph EVM and the XMR EVM are target architecture. The contracts deployed today run on Base Sepolia with ETH gas.

Architecture

	SERAPH EVM	XMR EVM
Gas	SER	XMR, or wXMR bridged from the base layer
Role	General-purpose contracts: DeFi, lending, launchpad, NFTs	Settlement hub with tight base-layer coupling
Precompiles	Universal (0x20 onward)	Privacy-preserving logic and cross-EVM calls
Cross-chain	Through the cross-EVM bridge	Through the cross-EVM bridge

Seraph EVM

- **Base fee token.** SER.
- **Compatibility.** Bytecode-compatible with Cancun and Prague. Solidity, Vyper, and Yul contracts deploy unchanged.
- **Confidential extensions.** Developers import Seraph privacy libraries to shield variables, store encrypted state, and authenticate callers with zero-knowledge proofs.

XMR EVM

- **Gas in XMR.** Gas is counted in gas units and charged in atomic XMR (piconero) or in bridged wXMR.
- **Base-layer synchronization.** Its state is coupled to the base layer's Monero anonymity pool.
- **Cross-EVM calls.** Contracts can call into the Seraph EVM through the bridge.

Privacy precompiles

Both runtimes add precompiles in the reserved range 0x20 to 0x25.

ADDRESS	NAME	PURPOSE	GAS
0x20	PEDERSEN_COMMIT	Computes C = vH + rG over BN254	750
0x21	POSEIDON_HASH	Algebraic hash for ZK circuits	300
0x22	STEALTH_DERIVE	DKSAP derivation	1,200
0x23	BULLETPROOF_VERIFY	Verifies range proofs for v in [0, 2^64 − 1]	12,000
0x24	MERKLE_INSERT	Incremental Merkle tree insertion	2,500
0x25	NULLIFIER_CHECK	Checks and records nullifiers in state	800

> **Under review.** The original manual stated that these precompiles reduce cost by up to 90% compared with pure Solidity. No benchmark supports that figure yet. Remove it or publish the benchmark. The input and output encodings and error behavior also need to be documented for each precompile.

Confidential storage

Seraph introduces storage slots whose values are encrypted under a contract-level key. The contract holds a commitment and a ciphertext, not a plaintext balance. A state change requires a proof that the transition is authorized.

```
contract PrivateBalanceVault {
    mapping(address => bytes32) private encryptedBalances;

    function transferConfidential(
        bytes32 commitmentIn,
        bytes32 commitmentOut,
        bytes calldata proof,
        uint256[] calldata pubSignals
    ) external {
        require(
            IVerifier(verifier).verifyProof(proof, pubSignals),
            "Invalid proof"
        );
        // update encryptedBalances using commitmentOut
    }
}
```

Observers see that a state transition happened but cannot decode the amounts.

> **Under review.** Commitments in this example must be homomorphic if the update relies on a homomorphic balance check. Chapter 2 describes the open decision on commitment type.

Tooling compatibility

- **Foundry and Hardhat.** Standard `forge build` and `forge test`, with no compiler fork.

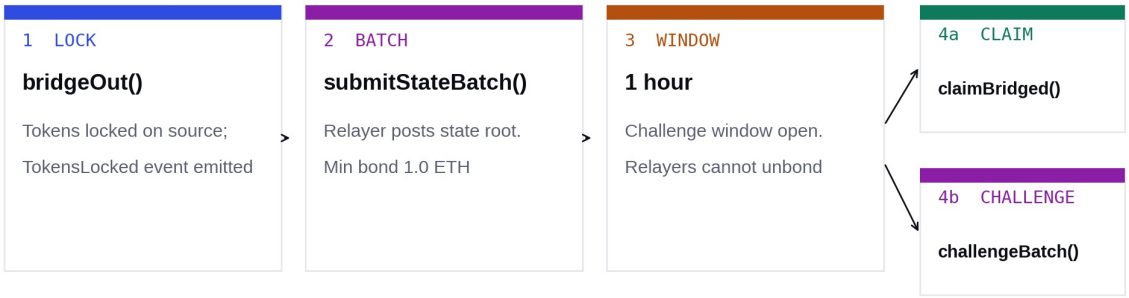
- **Ethers.js and Viem.** Standard JSON-RPC methods, including `eth_call`, `eth_sendRawTransaction`, and `eth_getLogs`.
- **Wallets.** Connect by adding the RPC URL and chain ID.
- **OpenZeppelin.** Inherits OpenZeppelin v5 libraries, such as ERC20, ReentrancyGuard, and Ownable.

Chapter 7: Bridges and cross-chain architecture

Design principles

Seraph connects three environments: the sovereign base layer, the Seraph EVM, and the XMR EVM. The design avoids trusted multisig federation bridges and uses an optimistic bridge with a challenge window.

> **Under review.** The trust model is stated in two places that do not fully agree. This chapter describes a guardian-controlled challenge path. Until the fraud-proof procedure in section 7.3 is specified, describe the bridge as guardian-assisted, not as fraud-proof.



Claim after the window closes unchallenged. A challenge marks the batch invalid and slashes the relayer bond.

```
Under review: the proof a challenge must present is not yet specified (Chapter 07).
```

Bridge lifecycle: lock, batch, challenge window, then claim or challenge

Lock, prove, and mint

Step 1: lock on the source chain

```
function bridgeOut(
    address recipient,
    uint256 amount,
    bytes32 destChainId
) external nonReentrant returns (uint256 nonce);
```

Tokens are locked in the bridge contract, a unique outbound nonce is assigned, and a `TokensLocked(nonce, sender, recipient, amount, destChainId)` event is emitted.

Step 2: optimistic batch submission

A bonded relayer packages outbound transactions into a Merkle state root and submits it on the destination chain.

```
function submitStateBatch(
    bytes32 sourceChainId,
    bytes32 stateRoot
) external whenNotPaused returns (uint256 batchId);
```

The relayer must bond at least 1.0 ETH (`MIN_RELAYER_BOND = 1 ether`). The submission timestamp opens a one-hour challenge window (`CHALLENGE_WINDOW = 1 hours`).

Step 3: claim

After the window closes without a challenge, the recipient calls `claimBridged`.

```
function claimBridged(
    uint256 batchId,
    bytes32 nullifier,
    address recipient,
    uint256 amount,
    bytes32[] calldata merkleProof
) external nonReentrant whenNotPaused;
```

The bridge checks the leaf, `Keccak256(nullifier || recipient || amount || localChainId)`, against the batch root. It records the nullifier to prevent replay, then unlocks or mints the tokens.

Bonds, challenges, and slashing

1. **Bonding.** A relayer bonds at least 1.0 ETH through `bondRelayer()`.
2. **Unbonding.** `unbondRelayer(amount)` checks that no batch from that relayer is inside an active challenge window.
3. **Challenges.** The guardian or an automated watcher calls `challengeBatch`. The batch is marked challenged, claims from it are blocked, and the relayer's bond is slashed. The slashed amount goes to the challenger.

> **Under review.** Two things must be settled before this section can be called fraud-proof.
>
> - **The proof.** What data a challenge contests, and how the contract verifies the challenge on-chain.
> - **Access control.** `challengeBatch` takes a challenger address as a parameter. Restrict who can call it and who the reward goes to.

A one-hour window is short for high-value flows. Longer windows or validity proofs for large transfers are under consideration.

Emergency pause

The bridge has a guardian-controlled pause. While paused, `bridgeOut`, `submitStateBatch`, and `claimBridged` revert with `BridgePaused()`. Relayer bonds stay in escrow until the guardian completes review and unpauses. Chapter 18 covers the procedure.

wXMR

wXMR is an ERC-20 representation of Monero held in base-layer vaults. It uses 18 decimals and can be used in lending markets, liquidity pools, and gas payments through the GasRelayer.

> **Under review.** The reserve model, custody arrangement, and reserve verification for wXMR are not yet documented. Users should not treat wXMR as fully backed until they are.

Chapter 8: Relayers and gas abstraction

The gas problem in confidential systems

Normally a user must hold a native gas token in a public wallet. This creates a privacy leak. A user can receive funds in a stealth address, but to move them, the address needs gas. The gas funding transaction connects the stealth address to a public wallet or an exchange, which defeats the privacy the stealth address provided.

Seraph removes this link with gas relayers (`GasRelayer.sol`). A user can execute transactions without holding native gas, and the fee is paid in SER or XMR from a pre-funded voucher.

How the relayer works

1. The user signs an intent, including a voucher nullifier, and sends it to the relayer.

2. The relayer submits the transaction on-chain and pays native gas.
3. The contract executes the user's payload.
4. The contract reimburses the relayer in SER or XMR from the voucher pool.

Funding a voucher

```
function fundGasVoucher(
    address feeToken,
    uint256 amount,
    bytes32 voucherCommitment
) external nonReentrant;
```

`feeToken` is SER or wXMR. `voucherCommitment` is a hash of an unspent secret scalar. Tokens are held in pooled escrow.

Gas-abstracted execution

```
function executeWithGasAbstraction(
    bytes32 voucherNullifier,
    address feeToken,
    uint256 feeAmount,
    address target,
    bytes calldata data
) external nonReentrant returns (bytes memory result);
```

The contract checks that the voucher nullifier has not been used and that the pool holds enough fee tokens. It marks the nullifier used, pays the relayer `feeAmount`, and calls `target` with `data` on the user's behalf.

Target filtering

To stop attackers from using the relayer to drain its own assets, the contract rejects some targets.

```
error ForbiddenTarget();

if (
    target == address(0) ||
    target == address(this) ||
    target == address(serToken) ||
    target == address(wxmrToken)
) revert ForbiddenTarget();
```

Reentrancy locks prevent recursive drains. An attacker cannot call the token directly to move the relayer's balance.

Gas rate settings

The rate between the native gas token and the fee token is set by a governance-controlled function.

```
function setTokenGasRate(address feeToken, uint256 rate) external onlyOwner;
```

Users are designed to pay a small overhead, target under 3% above raw gas cost.

> **Under review.**
>
> - **Which gas token.** On Base Sepolia the relayer pays ETH gas. The Seraph EVM is designed to use SER gas. The gas section must state which chain it describes.
> - **Who sets the rate.** The setter is restricted to the owner. The text describes governance. Confirm the authority and document any bounds on changes.
> - **The overhead figure.** The under-3% figure is a target. Measure it before publishing it as a result.
>
> The original heading "Gas Rate Oracles" described no oracle. This section is titled "Gas rate settings" until an oracle exists.

Chapter 9: Sequencer architecture and MEV shielding

The MEV problem

On public smart contract networks, transactions wait in a visible mempool before inclusion. Searchers and bots can:

1. **Front-run.** Detect a profitable trade and send a transaction ahead of it by paying more gas.
2. **Sandwich.** Surround a user's swap with a buy and a sell to extract slippage.
3. **Censor and snipe liquidations.** Reorder transactions to trigger liquidations or exclude privacy transactions.

In a privacy protocol, metadata leaking before execution can undo privacy guarantees. Protecting ordering is therefore part of the privacy design.

Encrypted mempool

Seraph's design encrypts transaction payloads with a threshold public key, so that no single party can read a transaction before its order is fixed.

1. **Client encryption.** The payload (recipient, function selector, parameters, value) is encrypted under a threshold public key. Relayers see only the gas bid and the sender's validity proof.
2. **Order commitment.** Sequencers commit to an ordered batch, H_batch = Keccak256($C_1 \| C_2 \| \ldots \| C_k$). Once the hash is posted, the order cannot change.
3. **Threshold decryption.** A committee of n nodes releases decryption shares. Once t of n shares are published (for example, 5 of 7), the batch decrypts.
4. **Execution.** The batch executes in the committed order.

Because the trade parameters are hidden until the order is fixed, the attack patterns above become much harder to run. The design reduces front-running and sandwiching. It does not make them impossible under every assumption, because it depends on the committee and on correct threshold key handling.

Protection by attack vector

VECTOR	PUBLIC EVM	SERAPH DESIGN
DEX sandwich	Searchers front- and back-run slippage	Swap amounts and paths stay inside the ciphertext until the order is fixed
Liquidation priority	Bots compete on gas priority	Batches execute in committed order without priority auctions
Oracle front-running	Bots act on known price updates	Price updates and user transactions settle in the same batch
Censorship	Validators can exclude privacy transactions	Validators cannot tell deposits, transfers, and swaps apart

Sequencer rotation (target design)

- **Epoch rotation.** A primary sequencer is chosen every 100 blocks from bonded SER stake.
- **Liveness.** If a sequencer misses its batch window, a standby takes over and the inactive sequencer's stake is slashed.
- **Equivocation.** If a sequencer signs two conflicting batch orders for the same height, an automated fraud proof slashes its full stake.

Under review.

- **Committee.** The distributed key generation, committee membership, key rotation, and recovery procedure are not yet specified.
- **Liveness.** The behavior when the threshold is not met is not yet specified. A batch that cannot be decrypted must not halt the chain without a defined recovery path.

- **Ordering rule.** The text says batches are ordered “first seen.” Define who decides the order and how.
- **Phase relationship.** Early phases use authority sequencing (Chapter 5). Confirm which parties run sequencing in each phase.

Chapter 10: Wallets, scanning, and key recovery

Wallet ecosystem

The Seraph wallet is non-custodial and runs on several platforms.

- **Desktop** (macOS, Linux, Windows). Native application with local proving and optional full-node sync.
- **Web and browser extension.** Lightweight, with WASM and WebGPU proving.
- **Mobile** (iOS, Android). Touch-optimized, with biometric key protection and optional delegated scanning.

Under review. Delegated scanning gives a third party the scan key. The privacy and trust trade-offs must be documented before the feature ships, including what the scanning party can learn.

Key hierarchy

All keys derive from one 24-word BIP-39 seed phrase.

KEY	DERIVATION (AS SPECIFIED IN THIS CHAPTER)	FUNCTION	HOLDER
Seed	24-word BIP-39 phrase, path m/44’/9999’/0’/0/0	Root of all keys	User only
Master spend key	32 bytes from the seed	Authorizes spends	User only
Incoming view key	H_s(“seraph_viewkey_derive” ‖ spendKey)	Scans and decrypts incoming notes	Wallet, delegated scanner if enabled
Outgoing view key	H_s(“seraph_tax_disclosure” ‖ spendKey)	Signs disclosure packages	User, shared with an auditor if chosen
Stealth public key	H_s(“seraph_pubkey_derive” ‖ spendKey)	Identity for stealth addresses	Public

Under review.

- **Hash notation.** Chapter 3 derives the view key with Keccak256. This table uses H_s. Use one definition throughout.
- **Scan key.** The Seraphis design separates a scan key. This hierarchy has no separate scan key. Decide whether one is needed.
- **Coin type.** The BIP-44 coin type 9999 is not a registered value. Confirm it before release.

Scanning

Commitments on the chain are blinded, so a wallet must scan new blocks to find its notes.

```
        const scannedNotes = scanNotesForOwner(encryptedNotes, incomingViewKey);
```

Scanning uses trial decryption. For each new note, the wallet decrypts the memo with its incoming view key and the note’s public blinding, then checks the first four bytes against the magic value `SERP` (0x53455250). A match means the note is the wallet’s. A mismatch means it belongs to someone else.

A matched memo has this layout:

```
[4 bytes "SERP"] ‖ [8 bytes amount, big-endian] ‖ [1 byte AssetId] ‖ [UTF-8 memo]
```

Under review. The memo cipher is described as an XOR with a view-key-derived value and no authentication. XOR without a message authentication code cannot detect tampering, and a 4-byte magic gives weak integrity. The recommended change is an authenticated encryption scheme (AEAD) with a documented key derivation function and nonce source. The 17-test Rust suite references a stream cipher, so confirm the implementation before updating this section.

Compliance disclosures

Users can generate disclosure packages for accountants or tax authorities.

```
const disclosure = generateComplianceDisclosure(spentRecords, outgoingViewKey);
```

A package lists the spent nullifiers, the amounts, the destination addresses, and the timestamps and transaction hashes.

Under review. The package is currently authenticated with a tag computed with the outgoing view key. Anyone who holds that key can create a valid package, so an auditor cannot verify that a package came from the user. The planned fix is to sign packages with a separate signing key whose public half is published. Until then, describe the package as a record the user produced, not as a signed statement.

Backups and recovery

1. **Seed phrase.** The 24 words recover all past, present, and future funds and history.
2. **Encrypted vault file.** Desktop and web wallets can export an AES-GCM-256 encrypted file with cached Merkle paths and labels, to restore quickly without a full rescan.
3. **Paper key.** A printable backup containing the master spend key and incoming view key in QR form.

Warning. The paper key contains the master spend key. Store it offline in a secure place, and do not photograph it or store it in cloud storage. Anyone who sees it can spend your funds.

Under review. The key-derivation function used to encrypt the vault file from the user's password is not yet documented.

Security practices

- Do not place spend keys in environment variables or plaintext configuration files. The SDK example in Chapter 12 uses an environment variable for brevity. Production systems should use a secrets manager or hardware-backed signing.
- Keep keys in memory as typed byte arrays and never write them to logs or unencrypted storage.
- Verify contract addresses against the deployment registry before signing.

Chapter 11: First-party privacy modules

Seraph includes first-party applications that run on the confidential EVMs.

Under review. This chapter describes three modules: CollateralVault (xUSD), UsageVault, and InvoiceRegistry. The public site lists five first-party modules. Confirm the list and the status of each before release.

MODULE	PURPOSE	STATUS
CollateralVault	Mint the xUSD stablecoin against SER or XMR collateral	Design, under review
UsageVault	Unlinkable usage tickets for AI agent compute billing	Testnet contract
InvoiceRegistry	Stealth invoicing and encrypted settlement messages	Design, under review

xUSD and CollateralVault

xUSD is a confidential, over-collateralized dollar-pegged token. Users lock SER or XMR and mint xUSD.

Minimum collateral ratio. Collateral value divided by debt must be at least 150%.

```
collateral_ratio = (collateral × price_USD) / debt_xUSD ≥ 1.50
```

Users mint with `depositAndMint`, and burn with `burnAndWithdraw`, which unlocks collateral while keeping the ratio above 150%. Once minted, xUSD can be shielded and transferred through the privacy pool.

> **Under review.** The following are not yet specified:
>
> - **Price source.** The oracle that supplies `price_USD`.
> - **Liquidation.** The threshold, the penalty, and the process when a position falls below 150%.
> - **Oracle failure.** What the system does when the price feed is stale or unavailable.
> - **Amount visibility.** Whether collateral and debt amounts are public. The privacy claim depends on this.
>
> Until these are defined, the 150% ratio is a parameter, not a guarantee.

UsageVault and AI agent billing

Autonomous agents need frequent, low-value payments without exposing their budgets. UsageVault issues one-time usage tickets.

1. The client deposits credits with a commitment, H(amount, secretPreimage).
2. The client sends a one-time ticket off-chain to a compute provider. The ticket contains the amount and the preimage.
3. The provider redeems the ticket on-chain. The contract checks H(amount, secret) against the commitment and records the ticket nullifier, H(secretPreimage).
4. The provider withdraws earnings with `claimEarnings`.

```
import { createCreditDeposit, UsageVaultClient } from "@seraph-protocol/sdk";

const secretPreimage = randomBytes32();
const ticket = createCreditDeposit(100n * 10n ** 18n, secretPreimage);

const client = new UsageVaultClient(vaultAddress);
const tx = client.encodeDeposit(ticket.commitment, 100n * 10n ** 18n);
```

> **Under review.**
>
> - **Visible deposit amounts.** The deposit transaction publishes the amount and the depositor. The claim that usage is “unlinkable” applies to the tickets, not to the deposit. State that clearly.
> - **Bearer tickets.** A ticket is a bearer instrument. If it leaks, anyone can redeem it. The design needs an expiry, a refund path, or both.

InvoiceRegistry

InvoiceRegistry supports merchant invoicing and peer-to-peer business payments.

1. **Create invoice.** The merchant publishes an invoice bound to a stealth destination. The memo is encrypted under the buyer’s view key.
2. **Settle.** `settleInvoice(invoiceId, paymentNullifier)` records the payment nullifier so it cannot be reused across invoices.
3. **Messaging.** `sendEncryptedMessage(recipientPubkey, ephemeralPubkey, ciphertext)` posts an encrypted message to a recipient’s public key, for trade negotiation and payment notices.

> **Under review.**
>
> - **Payment proof.** `settleInvoice` does not take proof of the amount, asset, or recipient. As written, it cannot confirm that the invoice was paid. Add a proof or name the attestor.
> - **Key exchange.** The buyer’s view key must reach the merchant before the memo can be encrypted. Describe how.
> - **Messaging spam.** Messages have no fee or rate limit in the current design. Decide how to prevent spam.

Chapter 12: Ecosystem, DEX, and CEX integrations

Why integrations need special handling

DeFi liquidity and exchange listings expect standard ERC-20 behavior. Tokens with transfer taxes or non-standard balance logic can cause failed routes, slippage mismatches, or unexpected fees. Seraph handles this with a tax-exemption registry and integration guidance for automated market makers and custodians.

The multi-hop problem

A swap from SER to USDC can pass through several pools and a router. If a 1% sell tax applied to every hop, the user would pay tax several times, or the router would fail on unexpected received amounts.

Tax exemption registry

`SERAPH.sol` keeps a mapping of exempt addresses, set by the treasury.

```
mapping(address => bool) public taxExempt;
event TaxExemptionUpdated(address indexed account, bool isExempt);

function setTaxExempt(address account, bool isExempt) external onlyTreasury {
    taxExempt[account] = isExempt;
    emit TaxExemptionUpdated(account, isExempt);
}
```

Warning. The intended rule is: a sell is a transfer into a registered pool from a non-exempt address. Under this rule, a user selling into a whitelisted pool is taxed once, and router legs are not taxed.

Under review. The transfer-hook condition as written in an earlier draft checks `taxExempt` on the recipient. If the recipient is a whitelisted pool, that check would exempt the sell. Do not whitelist a live pool until the condition is fixed and a test shows that a sell into a whitelisted pool is taxed exactly once and that multi-hop routes are taxed once.

Whitelisting a DEX pool

1. When a new Uniswap v3 pool or Aerodrome gauge is deployed, the treasury, under DAO governance, sets:
   - `setTaxExempt(poolAddress, true)` only when the pool is designed to be exempt, and
   - `setTaxExempt(routerAddress, true)` for the router.
2. Transfers between routers, pools, and liquidity providers are not taxed.
3. Sells by end users through public markets are taxed.

Under review. Step 1 says to exempt the pool. Exempting the pool and taxing sells into it cannot both hold under the condition as written. Decide whether pools are exempt, whether routers are exempt, and whether sells are taxed by pool address, then update this section to match.

Centralized exchanges

Exchanges use hot and cold wallets with internal sweeps between omnibus addresses.

1. The exchange gives its custody addresses to the treasury before listing.
2. The treasury adds them to `taxExempt`.
3. Internal sweeps are not taxed, and deposits and withdrawals match exact balances.

Under review. The treasury can exempt any address. Publish the list of exempt addresses and the process for adding one, so holders can verify it.

SDK example: AI agent payments

```
import { deriveWalletKeys, computeNoteCommitment, UsageVaultClient,
         createCreditDeposit, AssetId } from "@seraph-protocol/sdk";

const agentSpendKey = Buffer.from(process.env.AGENT_SPEND_KEY!, "hex");
const keys = deriveWalletKeys(agentSpendKey);

const computeCost = 10n * 10n ** 18n; // 10 SER
const secretPreimage = crypto.randomBytes(32);
const ticket = createCreditDeposit(computeCost, secretPreimage);

const vaultClient = new UsageVaultClient(USAGE_VAULT_ADDRESS);
const tx = vaultClient.encodeDeposit(ticket.commitment, computeCost);
```

Warning. This example reads a spend key from an environment variable for readability. In production, use a secrets manager or hardware-backed signing, and do not log the key. The example also uses `randomBytes32` and `crypto.randomBytes` interchangeably. Use one helper in the final code.

RPC methods

Seraph nodes expose JSON-RPC 2.0 with these custom methods.

METHOD	PURPOSE
`seraph_getLastRoot()`	Latest Merkle root of the commitment tree
`seraph_isKnownRoot(root)`	Whether a 32-byte root is in the 100-root ring buffer
`seraph_getNotesByRange(startLeaf, endLeaf)`	Encrypted memo payloads for client-side scanning
`seraph_estimateGasVoucher(feeToken, gasLimit)`	Voucher amount needed for gasless execution

Clients can subscribe to privacy events over WebSocket.

```
const ws = new WebSocket("wss://rpc.seraphprotocol.tech/ws");
ws.send(JSON.stringify({
  jsonrpc: "2.0",
  id: 1,
  method: "eth_subscribe",
  params: ["logs", { address: PRIVACY_POOL_ADDRESS }]
}));
```

A proof that references a root older than the last 100 root updates will be rejected. Wallets should refresh the root before proving.

Chapter 13: SER tokenomics, supply, and burns

Under review. The allocation in this chapter does not match the public site. Read section 13.2 before relying on any allocation figure.

Fixed total supply

SER is the native utility and governance token. Its supply is capped at the contract level.

```
MAX_SUPPLY = 100,000,000 × 10^18 wei  (100 million SER)
```

```
uint256 public constant MAX_SUPPLY = 100_000_000 * 1e18;

function mint(address to, uint256 amount) external onlyMinter {
    require(totalSupply() + amount <= MAX_SUPPLY, "SeraphToken: max supply exceeded");
    _mint(to, amount);
}
```

No administrator, multisig, or governance proposal can raise the supply above 100 million.

Allocation

> **Under review.** The original manual and the public site give different allocations. Both are shown below. The team will publish one table before the TGE. Do not rely on either table until then.

CATEGORY	ORIGINAL MANUAL (CHAPTER 13 DRAFT)	PUBLIC SITE (OCTOBER 2026)
Presale (strategic, private, public)	30%	Included in “Public sale + initial liquidity” (10%)
Protocol treasury and DAO	25%	Foundation / treasury: 15%
Liquidity and DEX bootstrapping	20%	Included in “Public sale + initial liquidity” (10%)
Core contributors and team	15%	Team and advisors: 20%
Ecosystem grants and research	10%	Ecosystem and community: 40%
Early contributors	Not listed	15%

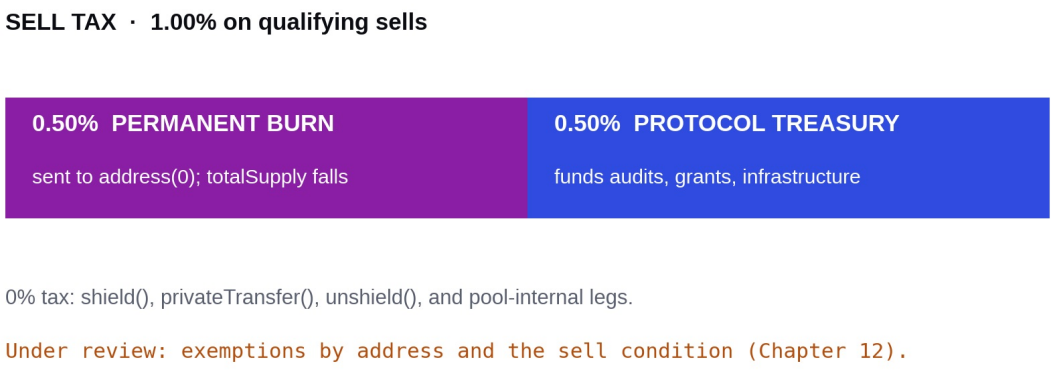
Under the original manual, presale tiers total 30 million SER, team allocations use a 12-month cliff and 36-month linear vesting, and the 20 million SER liquidity allocation unlocks at TGE. The team has not yet defined the treasury release schedule or the mechanism for liquidity.

Utility

1. **Gas.** Primary gas asset on the Seraph EVM.
2. **Collateral.** Locked in the privacy pool and CollateralVault to mint xUSD.
3. **Governance.** Holders vote on protocol parameters, precompile additions, and grants using private ballots (Chapter 16).
4. **Relayers and sequencers.** Relayers bond SER to earn routing fees on the bridge and to take part in sequencing.

Sell tax and burn

Sells of SER incur a 1.00% tax, with the following split.



Sell tax split: 0.50% burn and 0.50% treasury

- **Tax rate.** 1.00% (`SELL_TAX_BPS = 100`).
- **Burn.** 50% of the tax is permanently burned (`BURN_BPS = 5000`), sent to `address(0)` , which reduces `totalSupply()` .
- **Treasury.** The remaining 50% goes to the treasury.

The contract tests two invariants under random fuzz actions: supply conservation (`totalSupply() + totalBurned == totalMinted`) and the ceiling (`totalSupply() ≤ MAX_SUPPLY`).

Transfers into and out of the privacy pool are not taxed. Shielding, private transfers, and unshielding carry 0% tax.

> **Under review.** The sell condition and its exemptions are described in Chapter 12. Confirm the fix described there before relying on the sell-tax figures.

Fees and token utility

Fee rules are set in code and can be verified on-chain. For the fee architecture of launchpad tokens, see Chapter 15.

Chapter 14: Presale architecture and vesting

> **Status.** The presale contract is deployed to Base Sepolia at `0x2c4156e4bFE0bE6A4e683C7Be521E1cC10B798bE`. It is a testnet deployment. No mainnet sale has occurred.

> **Under review.** Presale parameters, tier sizes, and the allocation they sit inside are not final. See Chapter 13. The sale also needs legal review, including the KYC requirements, before a mainnet sale can run.

Design

The presale distributes early ownership across three tiers and uses vesting schedules to limit post-TGE selling.

Tiers (testnet parameters)

TIER	NAME	PRICE	ALLOCATION CAP	TGE UNLOCK	CLIFF	LINEAR VESTING	ACCESS
1	Strategic	$0.02	10,000,000 SER	25%	14 days	12 months	Whitelist and KYC
2	Private	$0.03	10,000,000 SER	20%	14 days	12 months	Whitelist and KYC
3	Public	$0.04	10,000,000 SER	100%	None	None	Open, no KYC

At full sale, the three tiers raise $200,000, $300,000, and $400,000, a total target of $900,000. A 0.50% presale fee (`FEE_BPS = 50`) is deducted on purchases.

Payment

Investors can pay with:

1. **USDC** (6 decimals) through `buyWithStablecoin(tierId, amount, usdc, referrer)` .
2. **USDT** (6 decimals) through `buyWithStablecoin(tierId, amount, usdt, referrer)` .
3. **ETH** (18 decimals) through `buyWithETH(tierId)` , or a direct transfer to the contract's `receive()` fallback.

> **Under review.**
>
> - `receive()` **purchases.** A plain ETH transfer buys tokens through the fallback. Decide whether this path stays. Each purchase path needs its own rules and tests.
> - **Repeat purchases.** The current design does not define how a second purchase from the same wallet updates its record. Define that before mainnet, so earlier allocations are never overwritten.
> - **USDC and USDT addresses.** The target-network addresses are pending. See the deployment registry.

ETH is converted using `ethUsdPrice` , a price set by the treasury.

```
tokens_purchased = (eth_amount × ethUsdPrice) / tier_price
```

Under review. The ETH/USD price is set by a privileged role. Publish how it is updated and bound its movement. Stablecoin conversions use 6-to-18-decimal scaling and do not lose precision.

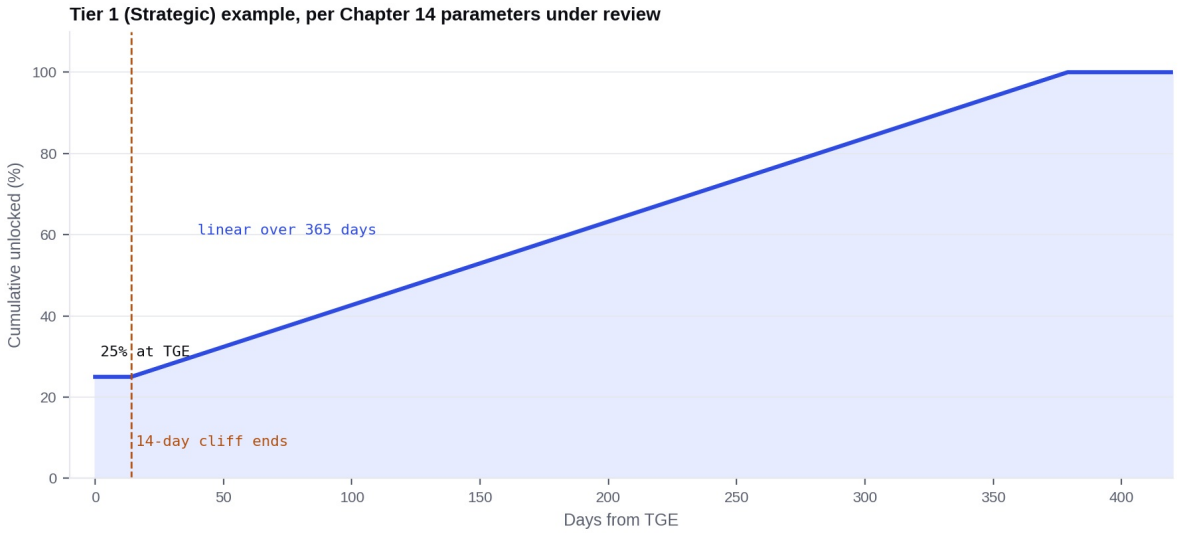
Vesting

For an investor with total allocation A, the initial claim at TGE is:

```
initial_claimable = A × unlock_bps / 10,000
```

After TGE, a 14-day cliff applies. No additional tokens unlock during the cliff. After the cliff, the remaining balance vests linearly over 365 days.

```
releasable(t) = (A − initial) × min(1, (t − (TGE + 14d)) / 365d) − already_released
```



Vesting curve for the Tier 1 example

The vesting curve for Tier 1 reaches full unlock at TGE + 379 days (14 days cliff plus 365 days linear).

Rounding

The test suite checks that after TGE + 379 days, released equals total to the single wei, so no dust is stranded in the contract.

Under review. This claim depends on the test suite. Re-run the rounding test after any change to vesting logic and before mainnet.

Compliance note

Tiers 1 and 2 require whitelisting and KYC. Public tier purchases are open and not KYC gated. The legal review should cover where each tier can be offered and whether any tier is restricted by jurisdiction. See the risk factors in the front matter.

Chapter 15: Launchpad mechanics and creator playbook

The Seraph launchpad (`Launchpad.sol`) lets creators, agents, and communities launch privacy-native ERC-20 tokens with bonding curves, fee splits, and automatic dividends.

Status. The launchpad is deployed to Base Sepolia at `0xe0349E4cC431329207e98C5205062776e058A846` . It is a testnet deployment.

Launch models

1. **Fixed pool (model 0).** A fixed token price from launch to graduation. Suited to predictable public allocations and airdrop claims.
2. **Bonding curve (model 1).** A constant-product market maker (x·y = k). The price rises with buys and falls with sells. A phantom virtual liquidity amount, Q_phantom, gives initial price stability without creator capital.
3. **Seraph pool (model 2).** A hybrid bonding curve integrated with the privacy pool. Buyers can shield received tokens into private notes in the same transaction.

Fees

- **Creation fee.** A flat 0.001 ETH paid at deployment, to the treasury, to discourage automated launch spam.
- **Platform fee.** A mandatory 1.00% on every buy and sell (`PLATFORM_FEE_BPS = 100`).
- **Creator fee.** Up to 2.50% on buys and sells (`buyFeeBps` , `sellFeeBps ≤ 250`).

Under these parameters, total trading fees can reach 3.50% on a trade.

> **Under review.** The creator splits use five destinations that sum to 10,000 basis points, and one of them is a platform share. The 1% mandatory platform fee is also listed separately. Confirm whether the platform fee is inside the creator's split or on top of it, and show a worked example.

Fee split

Creators divide their trading-fee revenue across five destinations:

```
creatorShare + buybackShare + dividendsShare + liquidityShare + platformShare = 10,000 bps
```

Presets:

1. **Creator-backed.** 100% to the creator.
2. **Diamond hands.** 100% to holder dividends.
3. **Deflationary.** 100% to buyback and burn.
4. **Auto-LP.** 100% to curve liquidity, to speed up graduation.
5. **Send to Seraph.** 100% to the protocol treasury.
6. **Custom.** Any split across the five destinations.

Dividends

Holder dividends use a global accumulator:

```
accDividendPerShare += (ΔDividendEth × 10^18) / EligibleTokenSupply
claimable(u) = balance_u × accDividendPerShare − rewardDebt_u
```

On a transfer, both parties settle their pending dividends, then the balances and reward debts update. The test suite checks that moving tokens through ten addresses never increases total claimable dividends.

> **Under review.** `EligibleTokenSupply` is not yet defined. Specify which addresses are excluded, such as the pool, burn address, and treasury.

Creator playbook

1. **Configure.** Choose metadata, ticker, initial supply (minimum 10^18 wei), fee caps (at most 2.5%), and a preset.
2. **Optional dev buy.** The creator may buy up to 10% of supply (`MAX_DEV_BUY_BPS = 1000`) in the same transaction as the launch. This keeps snipers from buying first.
3. **Launch.** Call `launch(params)` with the ETH value.
4. **Trade and claim fees.** Fees accumulate in a pull-escrow balance. Creators and the platform claim with `claimFees()` . A failing fee recipient cannot freeze trading.

5. **Graduation.** When real ETH reserves reach the graduation target, curve trading completes, reserved tokens unlock into the pool, and the token moves to full pool trading or an external AMM.

> **Under review.** The graduation threshold is not yet set. The target AMM and any liquidity-lock rules for graduated tokens are also undecided.

Chapter 16: Decentralized governance and private ballots

Governance stages

The protocol plans to move authority from a security council to a token-governed DAO. To avoid confusion with the roadmap phases in Chapter 19, governance stages are labeled A to D.

- **Stage A (genesis, current).** A fast-response Security Council, a 4-of-7 Gnosis Safe multisig, manages whitelists, emergency pauses, and presale contracts.
- **Stage B (mainnet launch).** A DAO treasury is created. Fee allocations and grants move to on-chain community votes.
- **Stage C (dual EVM launch).** Protocol parameters, including sell tax, gas rates, and precompiles, are governed by private ballots.
- **Stage D (sovereign).** Multisig emergency keys are revoked. Contract upgrades go through timelocked DAO governance only.

> **Under review.** Stage C and Stage D depend on the private ballot design below, which is not final.

Proposal lifecycle

1. **RFC.** Published to the governance forum for at least 7 days of review.
2. **SIP (Seraph Improvement Proposal).** Includes executable code, Foundry tests, and formal verification specifications.
3. **On-chain ballot.** Open for 5 days. Requires a quorum of 10,000,000 SER (10% of total supply).
4. **Timelock.** Passed proposals wait 48 hours before execution, so users can review the bytecode.

> **Under review.** Quorum requires a count of participation. Private ballots keep individual votes hidden, so define how participation is counted without revealing votes.

Private ballots

Public voting has three problems. Voters can face retaliation. Early votes by large holders sway others. Bribers can verify how an address voted before paying.

Seraph’s design uses shielded note nullifiers.

1. The voter proves that a note exists in the tree.
2. The voter derives a unique voting nullifier, so the note can vote once per proposal.
3. The voter casts a vote for an option, and the tally adds the note’s voting weight.

The design aims for three properties:

- **Weight integrity.** Voting power equals the unspent note’s value.
- **Double-vote prevention.** The voting nullifier is recorded, so the note cannot vote twice.
- **Unlinkability.** Validators and observers cannot link the ballot to a public address.

> **Under review.**
>
> - **Weight visibility.** Each vote increments the tally by the note’s weight, and that increment is public. Unique weights can reveal the voter’s holdings. Hide the weight, for example with fixed-weight ballots or a zero-knowledge tally.
> - **Snapshot rule.** The rules do not yet say when a note’s balance counts. Without a snapshot block, a user can buy, shield, vote, and unshield in one window. Define a snapshot.

- **Transfer after voting.** A note that votes and then moves to a new holder may be able to vote again with a new nullifier. The snapshot rule addresses this.
- **Coercion and bribery.** A voter can prove how they voted by revealing their voting secret. The design should state this limit plainly rather than claim full bribery resistance.

Chapter 17: Security model, threat vectors, and test coverage

Philosophy

Privacy protocols carry specific risks. A bug in a circuit or a state contract can cause financial loss and loss of user anonymity at the same time. Seraph combines several layers of defense.

1. **Formal reasoning.** Balance conservation and discrete-log assumptions are stated explicitly.
2. **Defensive contract design.** Reentrancy guards, pull-escrow for fees, and emergency pauses.
3. **Fuzz and invariant testing.** Stateful invariant checks across randomized transactions.
4. **Adversarial tests.** Rust base-layer tests and TypeScript DOM-injection tests.

Status. These are internal tests. They do not replace an external audit. See the audit status table below.

Threat matrix

#	THREAT	SCENARIO	MITIGATION IN THE DESIGN	TEST
1	Double spend	Spending a note twice in the same or a later block	Deterministic nullifiers, checked in constant time before state changes	`PrivacyPool.t.sol`, `adversarial_pentest.rs`
2	Cross-asset substitution	Using an XMR note in place of a SER note of equal value	Asset identifier bound into the commitment (see note)	`adversarial_pentest.rs`
3	Sybil dividend loop	Looping tokens through ten addresses to claim extra dividends	Reward-debt accounting, settled on transfer	`LaunchpadMultiActor.t.sol`
4	Flash-loan curve drain	Buy and sell in one transaction to exploit rounding	Round-trip sell value stays at or below buy cost	`DeepPentestLaunchpadPresale.t.sol`
5	Callback reentrancy	Re-entering a claim or buy from inside a `receive()`	ReentrancyGuard on state-changing functions	`DeepPentestLaunchpadPresale.t.sol`
6	Honeypot fee lockup	Creator sets a fee recipient that always reverts	Pull-escrow: failures never block trades	`LaunchpadMultiActor.t.sol`
7	Merkle tree overflow	Inserting more leaves than the tree holds	Capacity check, `nextLeafIndex < 2**TREE_DEPTH`	`PrivacyPool.sol`
8	Expired root	Proving against an old root to spend a spent note	100-root ring buffer; unrecorded roots revert	`PrivacyPool.t.sol`
9	Bridge fraud, front-running	Relayer submits a fake batch and unbonds before a challenge	Relayers cannot unbond while a batch is in its window	`CrossEVMBridge.sol`
10	Gas relayer drain	Passing the token as the target to drain the voucher pool	Forbidden-target filter	`GasRelayer.sol`
11	Front-running and MEV	Bots sandwich trades	Encrypted mempool, ordered before decryption	Design-level; tests pending (Chapter 9)

12	DOM injection	Token name containing a script tag, to steal keys in the web UI	Escaping of all on-chain strings before rendering	`security_pentest.test.mjs`

Under review.

- **Row 2.** The threat matrix says the asset identifier is hashed into the nullifier. Chapter 3 gives the nullifier without it. Resolve this before publication.
- **Row 11.** The “verifying test” for MEV is a design document, not an executable test. Mark it as design-level until tests exist.
- **Row 9.** This row describes the window protection. The proof a challenge must present is still open. See Chapter 7.

Test coverage (internal)

- **Foundry core suites.** 155 unit, scenario, and multi-actor tests.
- **Foundry stateful invariants.** 896,000 invariant checks with no reverts, covering pool solvency, the vault’s 150% collateral rule, and relayer escrow matching voucher claims.
- **Rust base layer.** 17 adversarial tests covering Merkle proofs, the view-key cipher, and multi-asset separation.
- **Node.js SDK and UI.** 43 tests covering ABI encoding, curve estimation, and live Base Sepolia RPC decoding.

Coverage percentages and line-level reports are not yet published. Add them before claiming full coverage.

Audit status

COMPONENT	AUDITOR	COMMIT	DATE	REPORT	STATUS
Token (`SERAPH.sol`)	None	Pending	Pending	Pending	Not audited
Privacy pool (`PrivacyPool.sol`)	None	Pending	Pending	Pending	Not audited
Spend verifier and circuits	None	Pending	Pending	Pending	Not audited
Bridge (`CrossEVMBridge.sol`)	None	Pending	Pending	Pending	Not audited
Gas relayer (`GasRelayer.sol`)	None	Pending	Pending	Pending	Not audited
Presale (`Presale.sol`)	None	Pending	Pending	Pending	Not audited
Launchpad (`Launchpad.sol`)	None	Pending	Pending	Pending	Not audited
Usage vault	None	Pending	Pending	Pending	Not audited
Collateral vault	None	Pending	Pending	Pending	Not audited
Invoice registry	None	Pending	Pending	Pending	Not audited
Base layer	None	Pending	Pending	Pending	Not audited

Proposed gate, under review: two independent audits and formal verification of the core circuits before mainnet funds are accepted.

Formal verification targets

Priority targets, in order:

1. The spend circuit: Merkle inclusion, nullifier derivation, commitment validity, and conservation.
2. Nullifier logic and double-spend prevention.
3. Balance conservation across the base layer and the EVM pool.
4. The sell-tax transfer condition.

Post-quantum note

The view-key derivations use hash functions that offer no post-quantum guarantees. The base-layer curves are also not post-quantum secure. Migration to post-quantum primitives is part of the long-term research track, with the STARK path in Chapter 4 as one option.

Bug bounty

The bug bounty pays for responsibly disclosed findings. Reports should include a reproducible Foundry or Rust test and go to `security@seraphprotocol.tech`, encrypted with the official PGP key.

SEVERITY	REWARD CAP	EXAMPLES
Critical	Up to $250,000	Direct theft of user funds, mass loss of anonymity across pool notes, full bridge drain
High	Up to $50,000	Unclaimed dividend drainage, temporary freezing that needs guardian action, bypass of the sell tax
Medium	Up to $15,000	Note linkage through side channels, rounding errors above 0.01%, non-critical denial of service
Low	Up to $3,000	Minor gas issues, documentation errors, reverts on edge cases

Under review. Scope, out-of-scope items, safe-harbor terms, the published PGP fingerprint, and payout currency are not yet defined. Publish them before the program accepts reports.

Chapter 18: Emergency procedures and circuit breakers

Seraph is designed to be decentralized over time, but early operations need fail-safes against undiscovered bugs, compiler issues, and bridge anomalies.

Circuit breakers

Two on-chain circuit breakers exist in the current design.

1. **`PrivacyPool.setPaused(bool)`.** Blocks deposits (`shieldFor`), private transfers (`privateTransfer`), and withdrawals (`unshield`).
2. **`CrossEVMBridge.setPaused(bool)`.** Blocks outbound bridging (`bridgeOut`), batch submission (`submitStateBatch`), and claims (`claimBridged`).

Pause coverage

CONTRACT	PAUSABLE	WHO CAN PAUSE	WHO CAN UNPAUSE	TIMELOCK	OTHER ADMIN POWERS
PrivacyPool	Yes	**Under review.** The original manual gives the treasury alone; the incident playbook gives the guardian council.	Same as pause	**Under review**	Verifier replacement (see below)
CrossEVMBridge	Yes	Guardian	Guardian	None stated	Challenge authority
SeraphToken	Not documented	Not documented	Not documented	Not documented	Tax-exemption setter
Presale	Not documented	Not documented	Not documented	Not documented	ETH/USD price setter
Launchpad	Not documented	Not documented	Not documented	Not documented	None documented
GasRelayer	Not documented	Not documented	Not documented	Not documented	Gas rate setter
UsageVault	Not documented	Not documented	Not documented	Not documented	None documented

CollateralVault	Not documented	Not documented	Not documented	Not documented	Oracle source (undefined)
InvoiceRegistry	Not documented	Not documented	Not documented	Not documented	None documented

Contracts without a documented pause function cannot be paused. Their recovery plan must be described before mainnet.

Access control

- **PrivacyPool.** Controlled through the treasury with the `onlyTreasury` modifier.
- **CrossEVMBridge.** Controlled by the guardian through `msg.sender != guardian` checks.
- **Guardian Council.** A 4-of-7 Gnosis Safe with hardware keys. Three core cryptographic engineers, two independent security auditors, and two community or validator representatives. Signers are spread across four legal jurisdictions, and all signing requires FIDO2 or WebAuthn hardware keys.

> **Under review.** The two independent security auditors on the council are signers, not auditors of the code. Disclose any conflicts of interest for council members.

Verifier replacement

The treasury can replace the spend verifier through `setSpendVerifier(newVerifier)` to fix a proving bug.

> **Warning.** Whoever controls the verifier controls the soundness of the privacy pool. A malicious or compromised replacement could allow counterfeit notes. Before mainnet, the team should choose one of three options and document it: a public timelock with advance notice, a verifier that is immutable per deployment (migration by redeployment), or a hard cap on what a single replacement can change.

Incident response

Phase 1: containment (target under 15 minutes).

1. Monitoring alerts on invariant deviations, such as an unexpected token drain or an invalid Merkle root.
2. The guardian council signs the pause transactions for `PrivacyPool` and `CrossEVMBridge`.
3. Shielding, private transfers, and unshielding revert. Bridge operations revert. Assets in the pool and bridge escrow stay in place.

> **Under review.** A 4-of-7 signing ceremony across jurisdictions may not finish in 15 minutes. Run a drill and publish the measured time before setting this target publicly.

Phase 2: root-cause analysis (target under 24 hours). Core developers and external auditors isolate the failing transactions and reproduce them in a Foundry fork test (`forge test --fork-url`). They then draft a patch, such as a verifier replacement or a logic hotfix.

Phase 3: resolution. If a proving bug exists, the treasury replaces the verifier. If a relayer submitted a fraudulent batch, the guardian calls `challengeBatch(batchId, treasury)` to slash the bond and cancel the batch.

Phase 4: unpause and post-mortem.

1. Verify the patch against the full test suites.
2. The guardian council signs `setPaused(false)` on both contracts.
3. Publish a public post-mortem within 48 hours, covering the timeline, root cause, economic impact, and permanent fixes.

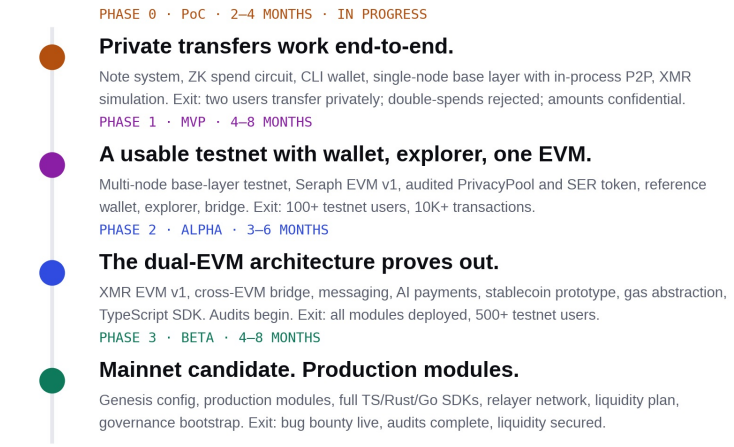
User communication and recovery

During a pause, the project will post status updates on its public channels and state which functions are affected. Users whose funds are frozen should expect to wait for the unpause. A compensation policy for losses from an exploit is not yet defined.

> **Under review.** Publish a recovery playbook covering: how users learn of a pause, what they should do with pending transactions, how claims are handled after a failed bridge batch, and how losses are assessed and compensated.

Chapter 19: Protocol roadmap and milestones

Under review. The public site and this manual use different phase breakdowns. The public roadmap below follows the site. The technical exit criteria follow the manual. A mapping table at the end shows the difference. One roadmap will be published.



Public roadmap: four phases from proof of concept to beta

Public roadmap

PHASE	NAME	DURATION (TARGET)	STATUS	EXIT CRITERIA
0	Proof of concept	2 to 4 months	In progress	Two users transfer privately; double spends are rejected; amounts stay confidential
1	MVP	4 to 8 months	Planned	100+ testnet users; 10,000+ transactions; audited privacy pool and SER token
2	Alpha	3 to 6 months	Planned	All modules deployed; 500+ testnet users; audits begin
3	Beta	4 to 8 months	Planned	Bug bounty live; audits complete; liquidity secured

Durations are targets, not commitments.

Technical phases in the manual

The manual uses five phases, numbered 0 to 4. The site's phases do not map one-to-one onto them.

MANUAL PHASE	FOCUS	MANUAL STATUS	SITE PHASE (FOR COMPARISON)
0: Base Sepolia testnet	Contracts deployed to Base Sepolia, documentation, test suites	Marked completed	Phase 0 proof of concept is in progress
1: Mainnet TGE, presale, privacy pool v1	Three-tier presale, TGE with a 100 million SER cap, DEX liquidity, Level 1 and 2 shielding for SER	Planned	Phase 1 MVP includes an audited privacy pool and SER token
2: Dual EVM testnet and bridge alpha	Seraph EVM and XMR EVM testnets, precompiles 0x20 to 0x25, optimistic	Planned	Phase 2 alpha includes the dual-EVM architecture

	precompiles 0x20 to 0x23, optimistic bridge, SDK v2		architecture
3: Sequencer decentralization and first-party modules	Encrypted mempool, sequencer rotation, xUSD, UsageVault, InvoiceRegistry	Planned	Phase 3 beta is a mainnet candidate with production modules
4: Sovereign base layer and next-gen cryptography	PoS and RandomX hybrid base layer, Seraphis and Jamtis, FCMP++, STARK upgrade	Research	Not shown on the site screenshot

Under review. The mapping table shows four specific mismatches:

- The manual marks Phase 0 as completed on Base Sepolia. The site lists the proof of concept as in progress.
- The site lists FCMP++ as part of the base layer in Phase 0. The manual lists it as a research horizon in Phase 4.
- The site describes the base layer as a Phase 0 proof of concept. The manual describes a sovereign base layer in Phase 4.
- The manual says CollateralVault, UsageVault, and InvoiceRegistry are deployed in Phase 0 and launched in Phase 3. Confirm the status of each.

Phase 0 scope (manual)

Deploy the contract suite to Base Sepolia (chain ID 84532): SeraphToken, PrivacyPool, Presale, Launchpad, CrossEVMBridge, GasRelayer, InvoiceRegistry, UsageVault, and CollateralVault. Publish the whitepaper with equations and diagrams. Build the client-side note vault and on-chain privacy station.

Exit: all contracts live, tested, and verified on Base Sepolia.

Proposed gate before mainnet

Under review. The team is considering a policy that no mainnet deployment accepts user funds until at least two independent audits and formal verification of the core circuits are complete, with that policy stated publicly. Confirm before publishing.

Phase 4 technical targets

- Base layer block times under 2 seconds, with sub-cent fees.
- No dependency on external bridges for the base layer’s own security.

Chapter 20: Glossary

Anonymity set. The set of notes that could have produced a given spend. In Seraph’s depth-20 pool this is every unspent note in the tree, up to 2^20 = 1,048,576.

Asset ID. A 32-bit identifier bound into note commitments to separate SER, XMR, and custom tokens inside one pool.

Auto-LP. A launchpad preset that sends trading fees into bonding-curve quote reserves to speed graduation.

Blinding factor. A random scalar r used in Pedersen commitments and note derivations to hide the committed value.

Bonding curve. A pricing formula (x·y = k) that sets price as a function of supply.

Bulletproofs++. A range-proof argument with logarithmic size that needs no trusted setup.

Circuit breaker. An on-chain pause (`setPaused`) that lets the guardian council freeze contract operations during an incident.

CLSAG. Compact linkable spontaneous anonymous group signature, historically used in Monero.

Commitment. A value that binds the committer to a value without revealing it. Seraph’s commitment scheme (Pedersen or Keccak-based) is under review. See Chapters 2 and 3.

Confidential storage slot. An encrypted EVM storage slot whose plaintext cannot be read without zero-knowledge authorization.

Dandelion++. A transaction-propagation protocol with stem and fluff phases, designed to hide the origin IP of a transaction.

DKSAP. Dual-key stealth address protocol. Lets a sender derive a one-time public key that only the recipient can recognize and spend from.

Dividend (launchpad). An ETH distribution to eligible token holders, tracked with a global accumulator.

Enote. An electronic note: a Seraphis-style output that separates ownership, value commitment, and nullifier generation.

FCMP++. Full-chain membership proofs. A research path that replaces small ring signatures with membership proofs over the whole ledger.

Fixed pool. A launchpad model with a fixed token price until graduation.

Gas abstraction. Paying execution fees in tokens such as SER or XMR without holding native gas.

Groth16. A pairing-based SNARK with constant-size proofs, verified on Ethereum at modest gas cost. Requires a trusted setup.

Guardian Council. A 4-of-7 Gnosis Safe multisig with hardware keys. Authorized to pause the bridge and, per Chapter 18, other contracts. Members: three core cryptographic engineers, two independent security auditors, and two community or validator representatives.

Homomorphic addition. A property that lets the sum of two commitments commit to the sum of the values.

Incoming view key. A private key that lets a wallet scan and decrypt incoming notes without spending authority.

Incremental Merkle tree. An append-only Merkle tree with constant on-chain storage per root. Insertion cost is under review: the manual states both O(log N) and O(1). Confirm before publishing.

Jamtis. The addressing standard designed for the Seraphis enote protocol.

Key image. A deterministic tag from a one-time private key, used to block double spends in Monero-style ring schemes.

MEV. Maximal extractable value: profit from including, excluding, or reordering transactions.

Mempool. The pool of pending transactions before inclusion in a block.

Nullifier. A deterministic 32-byte identifier derived from a spend key and note index. Revealing it marks a note spent without identifying the note.

Outgoing view key. A private key that authorizes compliance and audit disclosure packages.

Pedersen commitment. A commitment C = vH + rG, computationally binding and information-theoretically hiding.

Privacy continuum. Seraph’s four-level privacy model, from public EVM execution (Level 0) to confidential contract state (Level 3).

Pull-escrow. A design where balances accumulate in internal records and recipients claim them, so a failing recipient cannot block the contract.

Quorum. The minimum total voting weight for a proposal to pass. Currently 10,000,000 SER under Chapter 16.

Range proof. A zero-knowledge proof that a committed value lies in a valid interval, such as [0, 2^64 − 1].

R1CS. Rank-1 constraint system: a set of quadratic equations used to represent a circuit.

Seraph EVM. The general-purpose confidential EVM where gas is paid in SER.

Seraph Pool. The third launchpad model: a hybrid bonding curve integrated with the privacy pool. [Expand with mechanics.]

Sell tax. A 1.00% fee on qualifying SER sells, split 50% burn and 50% treasury. The qualifying condition is under review. See Chapter 12.

Sequencer. A node that orders encrypted transactions into batches and commits to the order. See Chapter 9.

Shielding. Depositing public tokens into the privacy pool to create a private note.

Stealth address. A one-time public key derived so that only the intended recipient can spend from it.

TGE (token generation event). The date on which SER becomes transferable and vesting starts. Not yet set. Vesting for each tier is measured from this point.

Tax exemption mapping (`taxExempt`). An on-chain registry of addresses exempt from the sell tax, such as DEX routers and custody wallets.

Threshold cryptography. A key split across n parties, with any t of them required to decrypt or sign.

Tier. One of three presale allocations (Strategic, Private, Public), each with its own price, cap, unlock, and vesting schedule. See Chapter 14.

Unshielding. Withdrawing a private note from the privacy pool to a public address.

Usage ticket. A one-time voucher used by an AI agent to pay a compute provider without linking the payment to the agent's main wallet.

Voucher. A pre-funded fee balance, identified by a commitment and spent by revealing its nullifier, used to pay gas through the GasRelayer.

wXMR. An ERC-20 token representing Monero held in base-layer vaults. Reserve model under review. See Chapter 7.

XMR EVM. The EVM environment where gas is paid in XMR or wXMR.

xUSD. A confidential over-collateralized stablecoin minted through the CollateralVault. Oracle and liquidation rules under review. See Chapter 11.

Chapter 21: Contributing, standards, and dev setup

Prerequisites

1. **Foundry.** Install with `curl -L https://foundry.paradigm.xyz | bash` , then run `foundryup` .
2. **Rust and Cargo.** Stable toolchain, version 1.80 or later. Install with `rustup` .
3. **Node.js.** Version 20 or 22.
4. **Git.**

Repository setup

```
git clone https://github.com/seraphprotocol/seraph.git
cd seraph

# 1. Smart contracts
export PATH="$HOME/.foundry/bin:$PATH"
forge build
forge test

# 2. Rust base layer
cargo build --manifest-path base-layer/Cargo.toml
cargo test --manifest-path base-layer/Cargo.toml

# 3. TypeScript SDK and UI
node --test sdk/typescript/test/*.test.mjs
```

> **Under review.** Confirm that the clone URL above is the canonical repository before publishing. Update it if the project lives under a different organization.

Smart contract standards

- Solidity `^0.8.24` , EVM target `cancun` .
- Inherit from OpenZeppelin v5 contracts.

- Every external function that accepts asset transfers uses `nonReentrant`.
- Fee claims and external transfers follow the pull-over-push pattern.
- Every contract change includes Foundry unit tests and fuzz invariant tests.

Base layer standards

- Format with `cargo fmt --check`, and pass `cargo clippy` with zero warnings.
- Changes to cryptographic commitments or nullifier hashing must add multi-asset isolation tests in `base-layer/tests/adversarial_pentest.rs`.

Frontend and SDK standards

- Keep runtime dependencies minimal.
- Sanitize all on-chain string data with `lpEsc()` before inserting it into the DOM.
- Keep cryptographic keys as typed byte arrays (`Uint8Array`), and never log them or store them unencrypted in local storage.

Branches, commits, and pull requests

Branch names:

- `feat/feature-name` for new protocol capabilities.
- `fix/bug-description` for security or logic fixes.
- `docs/documentation-update` for documentation.

Commit messages follow Conventional Commits:

```
feat(core): add taxExempt mapping for DEX routers
fix(pool): attach whenNotPaused to shield and unshield
test(pentest): add multi-actor invariant tests for gas relayer
docs(manual): add chapter 18 on emergency procedures
```

Pull request gate:

- All Foundry tests pass with zero reverts.
- All 17 Rust base-layer tests pass.
- All 43 Node.js tests pass.
- Two approvals from core maintainers before merging to `main`.

Appendix: Bibliography

> **Under review.** Verify exact titles, venues, and years before publication.

- Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G. *Bulletproofs: Short Proofs for Confidential Transactions and More.* IEEE Symposium on Security and Privacy, 2018.
- Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C. *Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting.* EUROCRYPT, 2016.
- Groth, J. *On the Size of Pairing-based Non-interactive Arguments.* EUROCRYPT, 2016.
- Fanti, G., et al. *Dandelion++: Lightweight Cryptocurrency Networking with Formal Anonymity Guarantees.* 2018.
- Noether, S. *Ring Confidential Transactions.* 2015.
- EIP-197: Precompiled contracts for optimal ate pairing check on the alt_bn128 curve.
- Koe. Seraphis and Jamtis protocol papers, Monero Research Lab.